Geometric-Informed Neural Operator (GINO)

Architecture and JAX/Flax Implementation

SciML Team Discussion

March 13, 2026

Problem Setup

Goal
Predict surface pressure on a complex car mesh using a hybrid graph/grid approach.

Inputs:

- Car surface mesh vertices:

$$V = \{v_i \in \mathbb{R}^3\}_{i=1}^{N_v}$$

- Voxel query grid ($N_q = 32^3$):

$$Q = \{q_j \in \mathbb{R}^3\}_{j=1}^{N_q}$$

- Signed distance field:

$$\phi(q_j)$$

Output:

- Predicted pressure at mesh vertices:

$$p(v_i)$$

Challenge: Irregular mesh geometries require graph networks, but volumetric data requires efficient grid solvers.

Visual GINO Architecture

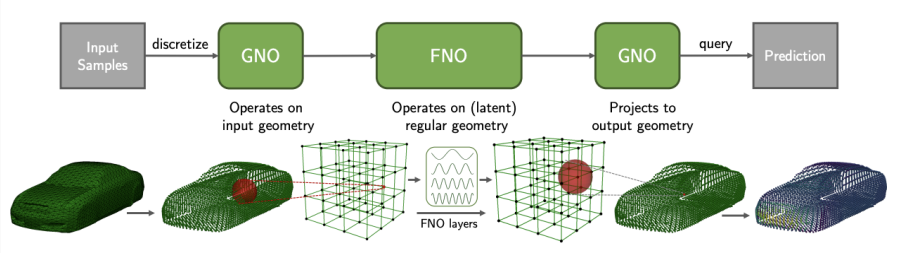

Figure 1: Schematic of the GINO pipeline: mapping irregular mesh inputs to a structured grid via GNO_{in}, spectral processing with FNO blocks, and projecting back to the mesh via GNO_{out}.

Model Architecture Components

Core Sub-Modules of GINO

The key architectural innovations making the solution efficient and resolution-invariant.

- **Graph Neural Operator (GNO):**
  - Solves the integration problem for irregular geometry by treating scattered data as graphs.
  - Handles transformations between mesh space and grid space via discrete convolution approximations.
- **Lifting and Projection Layers:**
  - Map low-dimensional geometric/physical features to and from high-dimensional latent space.
  - Typically implemented using simple MLPs or 1D convolutions for channel transformation.
- **Fourier Neural Operator (FNO):**
  - Processes grid-based data in the frequency domain.
  - Achieves highly efficient, resolution-invariant global convolutions on regular voxel structures.

GINO Architecture Overview

Forward Pipeline & Dimensionality
Detailed breakdown of data representation from input to output.

- **Pipeline:**

$$\text{Mesh} \xrightarrow{\text{GNO}_{in}} \text{Grid} \xrightarrow{\text{Lifting}} \text{FNO Blocks} \xrightarrow{\text{GNO}_{out}} \text{Mesh} \xrightarrow{\text{Projection}} \text{Output}$$

- **Transformations:**
  - Mesh vertices $V \to (N_v, 3)$
  - Grid points $Q \to (N_q, 3)$
  - Lifted representation $\to (N_q, 64)$
  - FNO processing $\to (N_q, 64)$
  - GNO output $\to (N_v, 64)$
  - Final prediction $p(v_i) \to (N_v, 1)$

GNO Input Block

Mapping Geometry to the Voxel Grid
Uses a Graph Neural Operator to map arbitrary mesh geometry to a structured voxel grid.

Inputs & Neighbor Search

$$y = \{v_i\}_{i=1}^{N_v}, \qquad x = \{q_j\}_{j=1}^{N_q}$$

$$\mathcal{N}(q_j) = \{v_i : \|v_i - q_j\| < r\}$$

Relative Coordinates & Positional Embedding

$$r_{ij} = v_i - q_j$$

$$\gamma(r_{ij}) = \left(\sin(2^k r_{ij}), \cos(2^k r_{ij})\right)_{k=0}^{L-1}$$

- Embedding dimension per edge: $6L \approx 384$

Kernel MLP and Aggregation

Continuous Kernel Computation

- **Kernel Network:** $384 \to 80 \to 80 \to 80 \to 3$ (with GELU activations)
- **Kernel Output:** $K(q_j, v_i) \in \mathbb{R}^3$

Message Passing / Aggregation

$$f(q_j) = \sum_{v_i \in \mathcal{N}(q_j)} K(q_j, v_i) \cdot x(v_i)$$

- Linear transform mode: $x(v_i) = v_i$
- Output features per query point: $f(q_j) \in \mathbb{R}^3$
- Stacked latent grid output: $(N_q, 3)$

Lifting and Latent Representation

Feature Expansion
Converts low-dimensional spatial features to high-dimensional latent space for spectral processing.

- **Input Shape:** $(B, N_q, 3)$
- **Channel MLP (Conv1D):** $3 \to 128 \to 64$
- **Reshape to Grid:**

$$(B, 64, 32, 32, 32)$$

Continuous latent field $f(q_j)$ is the input to the Fourier Neural Operator.

Fourier Neural Operator (FNO)

Spectral Convolution

- **Input Tensor:** $(B, 64, 32, 32, 32)$
- **Real FFT:** $\hat{u} = \text{rFFT}(u) \implies (B, 64, 32, 32, 17)$
- **Truncation:** Keep essential low-frequencies $\implies (B, 64, 16, 16, 9)$

Spectral Convolution Formula

Given spectral weights $W \in \mathbb{C}^{64\times64\times16\times16\times9}$:

$$\hat{y}_{b,c_{out},\ldots} = \sum_{c_{in}=1}^{64} \hat{x}_{b,c_{in},\ldots} \cdot W_{c_{in},c_{out},\ldots}$$

Inverse FFT: $y = \text{IFFT}(\hat{y})$ to return to spatial domain.

FNO Block Structure

Operations per Block:

1. Forward FFT
2. Mode truncation
3. Spectral convolution (in $\mathbb{C}$)
4. Inverse FFT
5. **Skip connection:**

$$x_{skip} = W_{1\times1\times1} * x$$

6. Channel MLP: $64 \to 64$
7. Activation (e.g., GELU)

Constraints

- **Number of Blocks:** 4 (Standard setup).
- **Resolution Invariance:** Block output is strictly preserved at $(B, 64, 32, 32, 32)$.

GNO Output Block: Grid to Mesh

Extracting the Latent Field

Maps the 64-channel latent voxel features back to the arbitrary mesh vertices by recalculating edge weights based on relative geometry.

1. Reverse Neighbor Search

For each target mesh vertex v_i, find voxel neighbors q_j:

$$\mathcal{N}(v_i) = \{q_j : \|v_i - q_j\| < r\}$$

2. Relative Geometry & Embedding

Recalculate positional embeddings for the new relative distances:

$$r_{ij} = v_i - q_j$$

$$\gamma(r_{ij}) = \left(\sin(2^k r_{ij}), \cos(2^k r_{ij})\right)_{k=0}^{L-1}$$

Embedding Shape: $\mathbb{R}^{384}$

3. Kernel Computation

Map the geometric embeddings to continuous kernel weights:

$$K(v_i, q_j) = \mathrm{MLP}(\gamma(r_{ij}))$$

MLP Channels: $384 \to 512 \to 256 \to 64$

4. Continuous Aggregation

Integrate the latent FNO features $f(q_j) \in \mathbb{R}^{64}$:

$$g(v_i) = \sum_{q_j \in \mathcal{N}(v_i)} K(v_i, q_j) \odot f(q_j)$$

Output Shape: $(N_v, 64)$

Projection and Missing Details (Training)

Projection Head
Decodes the 64 latent features to final scalar pressure.

$$64 \to 256 \to \text{GELU} \to 1 \implies p(v_i)$$

Training & Loss Setup

- **Loss Function:** Relative L_2 Norm loss over vertices:

$$\mathcal{L} = \frac{1}{B} \sum_{b=1}^{B} \frac{\|p_{\text{pred}}^{(b)} - p_{\text{true}}^{(b)}\|_2}{\|p_{\text{true}}^{(b)}\|_2}$$

- **Optimizer:** Adam or AdamW. Cosine annealing schedule with warmup.

JAX/Flax Implementation Plan

Core Components:

- Sinusoidal Positional Embeddings
- Spectral Convolution in $\mathbb{C}$ (using `jax.numpy.fft`)
- GNO Input/Output blocks
- Fully trainable FNO blocks & Projection head via Flax `nn.Dense`

Challenges & Solutions

- **Precomputing Neighbors:** Use SciPy/Open3D outside JIT loop for neighbor lists and distance masks.
- **Scatter Sums:** Apply `jax.ops.segment_sum` for graph message aggregation across segments.
- **Memory Management (32^3):** Leverage `jax.checkpoint` (gradient checkpointing) within FNO blocks to control VRAM scaling.